理想算子开发一面：排障与 Attention

> 技术面复盘 · 复现 / AI 协作 / 通信等待 / 类型分派 / FlashAttention。根据转写提炼并改写，公司与轮次沿用文件标注。

一、这一轮主要考什么

项目被连续追问“为什么”：为什么判定异常、为什么能在小环境复现、为什么比较逻辑会失效、为什么融合减少数据搬运。表达中是否能回到明确的数据流和状态变化，比复述项目摘要更关键。

二、源记录中可确认的提问

1. 选择一个最近完成的算子优化，介绍异常与定位过程。
2. 大规模任务中的问题，怎样缩小成本地有限资源上的复现？
3. 与 AI 协作开发时，怎样先确认方案、再实施并审核结果？
4. 是否遇到只对少量样例有效的特判？怎样发现并避免这种补丁？
5. 偶发通信等待中，控制标志值由谁设置、由谁更新、由谁判断？
6. 为什么用数据载荷承载某种控制状态？原设计的边界是什么？
7. 变更比较方式或位宽，具体改变的是哪一步？为什么满足原协议要求？
8. 迁移适配时，已有高性能算子库和新注册融合算子各承担什么作用？
9. 长跑失败与数据类型分派之间的关联如何定位？日志怎样采集？
10. 中间数值错误为何会延后表现成失败？如何找到首个异常点？
11. 从算法角度介绍 Attention 与 FlashAttention。
12. 为什么可以融合，原先需要的全局统计如何在分块中处理？
13. Tiling 究竟切哪几个维度，省掉了哪些中间数据读写？
14. 后续数据还没到达时，Online Softmax 怎样更新已有统计量？

三、整理补充：三项重点准备

把通信状态变化讲准确

用“初始状态、数据写入、状态发布、轮询观察、下一轮重置”逐步说明。每一步注明执行方、数据类型、可见范围和比较条件。转写在符号位、负零、位宽和比较方式上反复修正，不能用这些口述拼出确定的底层根因；需要实际版本代码与最小用例佐证。

从最后报错回溯到首个异常

先确认相邻算子的输入输出契约，包括 dtype、shape 和 layout。列出观察点，记录首次出现异常的中间值或分派路径；逐步缩小到一个操作。不能仅因为最后一个算子报错，就把根因归给它，也不能把所有长跑失败都解释为“误差积累”。

Attention 先写数学，再写数据流

练习写出标准注意力的输入、缩放、Softmax 与加权求和，再画 HBM 与片上存储间的分块流向。复习时手推两块数据的最大值和归一化统计如何合并，说明未保存完整中间矩阵时怎样得到最终输出。补充推导属于复习任务，不是记录中已经完成的回答。

四、AI 协作的面试表达

准备一个真实失败例子：AI 为什么产生只适配样例的方案、自己怎样发现、补了哪些不同输入或边界。最终能说明关键逻辑、审阅依据、回归结果和剩余限制，才构成完整交付过程。不要把工具给出的解释当成已验证事实。

五、来源与范围

来源：`理想 算子开发1面_转写.txt`。已去掉面试官姓名、个人履历、雇主、内部工作流细节、日期地点和流程沟通；未收录原对话中的具体收益或硬件比较结论。基础校准和公开参考入口见本目录《阅读说明与基础校准》。